

A Industry Oriented Mini Project Report
on
**Real Time Prognostics And Health Management Without Run To
Failure Data On Railway Assets**

Submitted to CMR ENGINEERING COLLEGE

In Partial Fulfillment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

NOMULA SRIVALLI (238R1A05DT)

REPANI PAVAN KUMAR (238R1A05DZ)

MD.IBRAHIM ALI SHARFI (238R1A05DN)

B. RAGHAVENDRA (238R1A05CD)

Under the Esteemed guidance of

Dr. G. Sumalatha

Associate Professor, Department of CSE



Department of Computer Science & Engineering

CMR ENGINEERING COLLEGE

UGC AUTONOMOUS

(Approved by AICTE, NEW DELHI, Affiliated to JNTU, Hyderabad Kandlakoya, Medchal Road, R.R.
Dist. Hyderabad-501 401)

2025-2026

CMR ENGINEERING COLLEGE

UGC AUTONOMOUS

(Accredited by NBA, Approved by AICTE NEW DELHI, Affiliated to JNTU, Hyderabad) KandlaKoya,

Medchal Road, Hyderabad-501 401

Department of Computer Science & Engineering



CERTIFICATE

This is to certify that the project entitled “**Real Time Prognostics And Health Management Without Run To Failure Data On Railway Assets**“ is a Bonafide work carried out by

NOMULA SRIVALLI	(238R1A05DT)
REPANI PAVAN KUMAR	(238R1A05DZ)
MD.IBRAHIM ALI SHARFI	(238R1A05DN)
B. RAGHAVENDRA	(238R1A05CD)

in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINEERING** from CMR Engineering College, affiliated to JNTU, Hyderabad, under our guidance and supervision.

The results presented in this Industry Oriented Mini Project report have been verified and are found to be satisfactory. The results embodied in this Industry Oriented Mini Project report have not been submitted to any other university for the award of any other degree or diploma.

Internal Guide
Dr. G. Sumalatha
Associate Professor
Department of CSE
CMREC, Hyderabad

Mini Project Coordinator
Mr. S Kiran Kumar
Assistant Professor
Department of CSE
CMREC, Hyderabad

Head of the Department
Dr. Sheo Kumar
Associate Professor
Department of CSE
CMREC, Hyderabad

External Examiner

DECLARATION

This is to certify that the work reported in the present Industry Oriented Mini project entitled “**Real Time Prognostics And Health Management Without Run To Failure Data On Railway Assets**” is a record of Bonafide work done by us in the Department of Computer Science and Engineering, CMR Engineering College, JNTU Hyderabad. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this Industry Oriented Mini Project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

NOMULA SRIVALLI	(238R1A05DT)
REPANI PAVAN KUMAR	(238R1A05DZ)
MD.IBRAHIM ALI SHARFI	(238R1A05DN)
B. RAGHAVENDRA	(238R1A05CD)

ACKNOWLEDGMENT

We are extremely grateful to **Dr. A. Srinivasula Reddy**, Principal and **Dr. Sheo Kumar**, HOD, Department of CSE, CMR Engineering College for their constant support.

We are extremely thankful to **Dr. G. Sumalatha**, Associate Professor, Internal Guide, Department of CSE, for her constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if We do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We are thankful to **Mr. S. Kiran Kumar**, Assistant Professor, CSE Department, Mini Project Coordinator for his constant support in carrying out the project activities and reviews.

We express my thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided me for every step.

NOMULA SRIVALLI	(238R1A05DT)
REPANI PAVAN KUMAR	(238R1A05DZ)
MD.IBRAHIM ALI SHARFI	(238R1A05DN)
B. RAGHAVENDRA	(238R1A05CD)

INDEX

CONTENTS	PAGE NO
ABSTRACT	i
LIST OF FIGURES	ii
LIST OF TABLES	iv
CHAPTER 1 - INTRODUCTION	
1.1 Introduction	01
1.2 Objectives of the Project	02
1.3 Existing System	03
1.4 Disadvantages of Existing System	03
1.5 Proposed System	03
1.6 Advantages of Proposed System	04
CHAPTER 2 - LITERATURE SURVEY	05
CHAPTER 3 - SOFTWARE REQUIREMENT ANALYSIS	
3.1 Problem Specification	07
3.2 Modules Description	07
3.3 Functional Requirements	08
3.4 Non-Functional Requirements	08
3.5 Feasibility Study	08
CHAPTER 4 - SYSTEM REQUIREMENTS	
4.1 Hardware Requirements	10
4.2 Software Requirements	10
CHAPTER 5 - SYSTEM DESIGN	
5.1 UML Diagrams	11
5.2 Data Flow Diagrams	17
5.3 System Architecture	18
CHAPTER 6 – CODING AND IMPLEMENTATION	
6.1 Methodology	19
6.2 Sample Code	30
CHAPTER 7 - TESTING	
7.1 Introduction to Testing	35
7.2 System Testing	35
7.3 Module Testing	36
7.4 Integration Testing	36
7.5 Acceptance Testing	36
CHAPTER 8 - OUTPUT SCREENS	38
CHAPTER 9 - CONCLUSION	43
REFERENCES	44

ABSTRACT

Prognosis is a challenging technology that aims to accurately predict and estimate the remaining useful life of a component or system in order to enhance its reliability and performance. Although prognosis research for predictive maintenance is a well-researched topic, practical examples of successful prognostic applications remain scarce. This is due to the lack of available run-to-failure data to build the prediction model as maintenance is usually conducted regularly to avoid significant defects. This paper proposes a novel prognosis method that can be applied to real-world railway maintenance planning without employing run-to-failure data. The key idea is that the fault severity assessment and approximate remaining time prediction are often all that is needed in order to plan maintenance. Firstly, using motor current signals, a degradation indicator on railway door systems is generated based on the dynamic time warping method to measure similarity between typical normal and faulty behaviour. Then, the K-means algorithm is applied to assess fault severity, followed by the representative time estimation for each level of fault severity. This estimation thus allows the remaining time prediction until reaching the critical fault severity level without using run-to-failure data. As a result, the proposed method enables predictive maintenance planning for railway door systems. In addition, the fault severity threshold can be updated by additional operational data, enabling the remaining time prediction to be more reliable. Furthermore, the proposed method can be applied to conventional railway assets and other electro-mechanical actuators as motor current signals are primarily available from the controller or motor drive without additional sensors. Next, the K-means clustering algorithm is applied to the degradation data to categorize the fault severity into discrete levels, providing a structured assessment of the system's current condition. For each identified fault severity level, a representative time estimation is calculated, enabling the prediction of the remaining time until the system reaches a critical fault threshold. This approach allows predictive maintenance scheduling without the need for historical run-to-failure data.

A key advantage of this method is its adaptability: as additional operational data becomes available, fault severity thresholds can be updated dynamically, which enhances the accuracy and reliability of remaining time predictions. This ensures that maintenance planning can continuously evolve with the operational history of the assets, improving both safety and cost efficiency.

LIST OF FIGURES

S.NO	FIG.NO	DESCRIPTION	PAGE NO
1	5.1.1	Class Diagram	12
2	5.1.2	Use Case Diagram	12
3	5.1.3	Sequence Diagram	13
4	5.1.4	Collaboration Diagram	14
5	5.1.5	Component Diagram	14
6	5.1.6	Deployment Diagram	15
7	5.1.7	Activity Diagram	15
8	5.2.1	Data Flow Diagram	17
9	5.3.1	System Architecture	18
10	8.1	In the above screen we got Tkinter Output Window. Now Click On the “Upload Dataset” button to upload the dataset to the application.	38
11	8.2	In the above we can see the dataset uploading.	38
12	8.3	In above screen loading and displaying dataset values.	39
13	8.4	Above is the 4 cluster graph where x-axis represents MOTOR current index and y-axis represents motor current values.	39

14	8.5	In above screen each colour circle represents one stage and the ending .	40
15	8.6	In above screen click on ‘Calculate Lifetime’ then it finds the Calculating Lifetime Using 10 clusters	40
16	8.7	In above screen in blue colour text we can see propose KMEANS score which will consider as difference between each cluster value	41
17	8.8	In above screen we are tuning KMEANS with various iterations and then in last we got KMEANS with lesser score and this less score KMEANS object is the optimized KMEANS.	42
18	8.9	In above graph x-axis represents algorithm names and y-axis represents SCORE and in both algorithm extension KMEANS got less score.	42

LIST OF TABLES

S.NO	TABLE.NO	DESCRIPTION	PAGE NO
1	2	Literature Survey	05
2	7.5	Acceptance Testing	37

1. INTRODUCTION

1.1 INTRODUCTION

Prognostics and Health Management (PHM) is an all encompassing technology that enables engineers to turn data and health states into information that can be used to increase the knowledge of a system and provide a strategy to maintain the system in its originally intended function. Whilst PHM originated in the aerospace industry, it is now being explored in many applications in industries such as manufacturing, automotive, railway, and heavy industry. There are many benefits to employing PHM, such as significantly reducing support and operating costs. For example, an unexpected one-day stoppage in the machinery industry may incur costs as high as up to 100,000 to 200,000 Euros . Furthermore, PHM also significantly increases safety as devastating accidents are more likely to occur from inadequate maintenance. One example of this is an incident on 10th May 2002 when a train travelling from London to Norfolk in the UK derailed at Potters Bar railway station, causing seven deaths and injuring over 70 people. The derailment was due to a points failure; one of the main factors being that the points had been poorly maintained. This incident demonstrates the fatal consequences that can arise from insufficient and inappropriate maintenance as well as the likely loss of public trust in the industry. Prognosis is a challenging technology that aims to accurately predict and estimate the remaining useful life (RUL) of a component or system in order to enhance its reliability and performance. RUL is the duration between the current time and the time at which the forecasted health level reaches a predefined failure threshold, which is when the system cannot continue fulfilling its intended functions. During the early stages of health monitoring technology, traditional applied technologies focused on detecting and isolating failures. As the demand for Condition-Based Maintenance (CBM) increased, the idea of using RUL as a prognostic failure prediction technique grew in popularity. Current prognostic approaches can be categorized into two major categories, namely physics-based models and data driven approaches. A typical physics-based models' prognostic strategy consists of dynamic models to perform the prediction function of the system's future state. Physics-based approaches provide technically comprehensive solutions that have been used widely to understand the failure progression. These models assume that an accurate mathematical model for degradation can be constructed from the first principle. Additionally, model parameters may be identified using empirical data obtained from specifically designed experiments. Then, the physics-based models can be used to determine the system's life usage by calculating the physics parameters for the system at that particular time. Once the current physics parameters have been identified, the model can predict future conditions based on historical conditions using stochastic techniques. Some of the most-used examples include fatigue crack propagation modeling

battery capacity modeling centrifugal pump degradation modeling thermal processing unit degradation pneumatic valve modeling and DC-DC converter system level degradation model. However, physics-based models largely rely on employing expert domain knowledge, and the models are also component-specific or system-specific, which means that they cannot be applied to other types of components or systems where the physics of failure mechanism differs. Moreover, industrial machinery systems contain many components interconnected with various uncertainties, which makes the physics-based modeling approach of limited value in predictive maintenance. On the other hand, data-driven approaches use historical run-to-failure (RTF) data to build a statistical, machine learning, or deep learning model. Data-driven approaches are divided into two categories: statistical models and machine learning models. Statistical approaches construct models by fitting a probabilistic model to the data without any engineering or physical principle knowledge. These approaches rely on statistical models and the observed data to support the prediction of the RUL. X.S. Si et al comprehensively reviewed the statistical approaches for RUL estimation. On the contrary, machine learning models attempt to recognize complex patterns and make a prediction based on key historical degradation information. Machine learning approaches are adaptable to situations where expert domain knowledge is unavailable. A typical prognosis method based on data-driven approaches is described. Firstly, a prediction model is built with RTF training samples acquired during machinery operations.

1.2 OBJECTIVE OF THE PROJECT

Prognosis is a challenging technology that aims to accurately predict and estimate the remaining useful life of a component or system in order to enhance its reliability and performance. Although prognosis research for predictive maintenance is a well-researched topic, practical examples of successful prognostic applications remain scarce. This is due to the lack of available run-to-failure data to build the prediction model as maintenance is usually conducted regularly to avoid significant defects. This paper proposes a novel prognosis method that can be applied to real-world railway maintenance planning without employing runto-failure data. The key idea is that the fault severity assessment and approximate remaining time prediction are often all that is needed in order to plan maintenance. Firstly, using motor current signals, a degradation indicator on railway door systems is generated based on the dynamic time warping method to measure similarity between typical normal and faulty behaviour. Then, the K-means algorithm is applied to assess fault severity, followed by the representative time estimation for each level of fault severity. This estimation thus allows the remaining time prediction until reaching the critical fault severity level without using runto-failure data. As a result, the proposed method enables predictive maintenance planning for railway door

systems. In addition, the fault severity threshold can be updated by additional operational data, enabling the remaining time prediction to be more reliable. Furthermore, the proposed method can be applied to conventional railway assets and other electro-mechanical actuators as motor current signals are primarily available from the controller or motor drive without additional sensors.

1.3 EXISTING SYSTEM

Prognostics refers to predicting the remaining useful life (RUL) and health of electronic or electromechanical components, such as railway gates, motors, and cutters, using sensor data like current signals. In existing systems, machine learning models are typically trained on historical run-to-failure datasets to estimate RUL. However, obtaining sufficient run-to-failure data is difficult because regular maintenance prevents complete component degradation. As a result, predictions can be inaccurate, reducing reliability and potentially risking human safety. Moreover, these methods are time-consuming due to extensive data preprocessing and model training, require additional sensors in some cases, and may not generalize well to new operating conditions, limiting their practical applicability.

1.4 DISADVANTAGES OF EXISTING SYSTEM

1. Predictions may be unreliable due to insufficient or noisy run-to-failure data.
2. Data preprocessing and model training take significant time, delaying results.
3. Some methods require extra sensors, increasing cost and complexity.
4. Models trained on specific datasets may not perform well under different conditions.

1.5 PROPOSED SYSTEM

To avoid this problem author of this paper proposing concept to capture LIVE data from Current motor and then this current data will be input to DTW (dynamic time wrapping) to measure similarity between old and new record and if current motor working properly then it way same similarity and if motor not working then there will be huge gap between previous and current similarity and this huge gap represents FAULTY motor. After predicting faulty motor author apply KMEANS algorithm on DTW values to generate 4 different clusters. Working motor cluster will have normal and flat curve and non-working motor cluster will have negative values. All clusters median time difference will be calculated to predict available life. For example first cluster last value will be subtracted from second cluster last value and the subtracted values will be considered as Lifetime. If available life time 0 or very less then that motor has to services or replace.

1.6 ADVANTAGES OF PROPOSED SYSTEM

1. The proposed method eliminates the need for run-to-failure datasets by using live motor current signals, making it practical for real-world applications.
2. It enhances prediction accuracy by integrating Dynamic Time Warping (DTW) with KMeans clustering, effectively handling nonlinear time-series data.
3. The system automates the entire workflow from data upload to clustering, reducing manual intervention and saving valuable time.
4. Preprocessing of datasets improves data quality, leading to more reliable and consistent analysis results.
5. By providing real-time feedback at each stage, the system ensures transparency and easier troubleshooting for users.
6. Utilizing DTW allows the method to compare sequences of different lengths and speeds, increasing robustness against data variability.
7. It supports condition-based maintenance strategies by accurately identifying patterns indicative of potential failures.
8. The proposed system reduces maintenance costs by enabling earlier detection of faults through efficient data-driven analysis.
9. The modular design facilitates easy integration with other machine learning models or prognostic algorithms in future upgrades.
10. The approach is adaptable to various industrial domains, enabling broad applicability without extensive customization.

2. LITERATURE SURVEY

S.NO	PAPER TITLE	AUTHOR	JOURNAL	PUBLISHED YEAR	ALGORITHMS USED	CONCLUSION
1	Data-Driven Prognostics for Railway Asset Health Monitoring	A. Kumar, S. Sharma	IEEE	2022	Random Forest, SVM	Used sensor data to predict failures without run-to-failure data, improving maintenance planning.
2	Remaining Useful Life Prediction Using Machine Learning in Railways	B. Singh, R. Patel	ACM Transactions on Intelligent Systems	2021	LSTM, ANN	LSTM models effectively estimated RUL using partial degradation data.
3	Hybrid Prognostics Approach for Railway Systems	C. Wang, D. Liu	Reliability Engineering Journal	2020	Kalman Filter, Bayesian Networks	Combined model-based and data-driven techniques to predict asset health without full failure history.
4	Deep Learning-Based Fault Detection in Railway Tracks	E. Johnson, M. Lee	Journal of Artificial Intelligence Research	2021	CNN, Autoencoders	Detected anomalies in track conditions using real-time sensor data.
5	Prognostics Without Failure Data Using Transfer Learning	F. Garcia, H. Kim	IEEE Access	2023	Transfer Learning, CNN	Leveraged pre-trained models to predict failures even with limited failure datasets.
6	Real-Time Health Monitoring of Railway Components	G. Taylor, P. Adams	Transportation Research Journal	2022	IoT, Time Series Analysis	Used real-time IoT data streams to monitor component health continuously.

S.NO	PAPER TITLE	AUTHOR	JOURNAL	PUBLISHED YEAR	ALGORITHMS USED	CONCLUSION
7	Machine Learning for Predictive Maintenance in Rail Systems	H. Zhang, L. Chen	Expert Systems with Applications	2020	Decision Trees, Gradient Boosting	Improved maintenance scheduling and reduced unexpected failures using data-driven models.
8	Anomaly Detection in Railway Systems Using Unsupervised Learning	J. Brown, K. Davis	IEEE Transactions on Industrial Informatics	2021	K-Means, Isolation Forest	Successfully identified abnormal patterns without requiring failure data.
9	Remaining Useful Life Estimation with Limited Data	M. Roy, S. Banerjee	Mechanical Systems and Signal Processing	2022	LSTM, Particle Filter	Achieved accurate RUL prediction using partial degradation data and hybrid models.
10	Digital Twin-Based Prognostics for Railway Assets	R. Wilson, T. Clark	IEEE Access	2023	Digital Twin, Simulation Models	Enabled real-time monitoring and prediction of asset health without full lifecycle failure data.

3.SOFTWARE REQUIREMENT ANALYSIS

3.1 PROBLEM SPECIFICATION

Predictive maintenance of motors and electro-mechanical systems is challenging due to the unavailability of run-to-failure datasets. Existing methods rely heavily on historical datasets to train machine learning models for predicting Remaining Useful Life (RUL). However, insufficient or incomplete data may lead to inaccurate predictions, increasing the risk of motor failure, unexpected downtime, and potential safety hazards. There is a need for a system that can monitor motors in real-time, detect faults early, and estimate remaining life without relying on extensive historical failure data.

3.2 MODULE DESCRIPTION:

Dataset Upload Module: Uploads prognostics dataset containing motor current values for further analysis. **Data Preprocessing Module:** Cleans and formats motor current signals to prepare them for DTW and clustering.

Live / Historical Current Signal Acquisition Module: Captures or reads current sensor data from motors for health monitoring.

DTW Similarity Analysis Module: Applies Dynamic Time Warping to measure similarity between past and current motor signals.

Fault Detection Module: Identifies faulty motors by detecting large deviations in DTW similarity scores.

K-Means Clustering Module: Clusters DTW values into four health states representing working and faulty conditions.

Lifetime Estimation Module: Computes remaining useful life using median time differences between cluster stages.

Optimized K-Means Tuning Module: Tunes K-Means parameters and iterations to minimize cluster distance score.

Performance Evaluation Module: Compares original K-Means and optimized K Means using cluster distance scores.

Visualization Module: Displays cluster graphs, lifetime stages, and score comparison charts.

User Interface Module (Tkinter GUI): Provides interactive buttons for dataset upload, analysis, and result visualization.

Dataset Upload Module: Uploads the prognostics dataset containing motor current values for further analysis. This module ensures that the data is properly imported and ready for preprocessing and subsequent analysis.

3.3 FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system must do.

1. Dataset Upload

- Users can upload motor current datasets of railway assets.
- System validates and stores data for analysis.

2. K-Means Clustering

- Groups asset conditions into multiple health states.
- Differentiates between normal and degraded conditions.

3. Lifetime Estimation

- Predicts remaining useful life without run-to-failure data.
- Uses cluster-based time differences for estimation.

3.4 NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe how well the system performs.

1. Security

- Protect motor data from unauthorized access.
- Ensure dataset integrity.
- Implement user authentication.

2. Scalability

- Handle multiple motors at once.
- Process large datasets efficiently.

3. Performance

- Real-time fault detection.
- Fast remaining useful life prediction.

3.5 FEASIBILITY STUDY

• ECONOMIC FEASIBILITY

The proposed motor prognostics system is economically feasible. The development leverages existing hardware such as motor sensors and standard computing resources, so no significant

additional investment is required. The software components, including DTW analysis, K-Means clustering, and visualization modules, are implemented using open-source tools, minimizing cost. The financial benefits, such as reduced maintenance costs, fewer unexpected motor failures, and extended equipment life, outweigh the development and operational expenses, making the project a sound economic investment.

- **OPERATIONAL FEASIBILITY**

Proposed projects are beneficial only if they can be turned out into information system. That will meet the organization's operating requirements. Operational feasibility aspects of the project are to be taken as an important part of the project implementation. This system is targeted to be in accordance with the above-mentioned issues. Beforehand, the management issues and user requirements have been taken into consideration. So there is no question of resistance from the users that can undermine the possible application benefits. The well-planned design would ensure the optimal utilization of the computer resources and would help in the improvement of performance status.

- **TECHNICAL FEASIBILITY**

Earlier no system existed to cater to the needs of 'Secure Infrastructure Implementation System'. The current system developed is technically feasible. It is a web-based user interface for audit workflow at NIC-CSD. Thus, it provides an easy access to the users. The database's purpose is to create, establish and maintain a workflow among various entities in order to facilitate all concerned users in their various capacities or roles. Permission to the users would be granted based on the roles specified. Therefore, it provides the technical guarantee of accuracy, reliability and security.

4. SYSTEM REQUIREMENT

4.1 HARDWARE REQUIREMENTS:

- Processor - Intel i3 (min)
- Speed - 1.1 GHz
- RAM - 4GB(min)
- Hard Disk - 500 GB

4.2 SOFTWARE REQUIREMENTS:

- Operating System - Windows10(min)
- Programming Language - Python 3.7.0
- Data Visualization Tools - Matplotlib
- Development Environment - VS Code/Jupyter Notebook

5. SYSTEM DESIGN

5.1 UML DIAGRAM:

The Unified Modelling Language allows the software engineer to express an analysis model using the modelling notation that is governed by a set of syntactic semantic and pragmatic rules. A UML system is represented using five different views that describe the system from distinctly different perspective. Each view is defined by a set of diagram, which is as follows.

- **User Model View**

- i. This view represents the system from the users perspective.
- ii. The analysis representation describes a usage scenario from the end-users perspective.

- **Structural Model view**

- i. In this model the data and functionality are arrived from inside the system.
- ii. This model view models the static structures.

- **Behavioural Model View**

It represents the dynamic of behavioural as parts of the system, depicting the interactions of collection between various structural elements described in the user model and structural model view.

- **Implementation Model View**

In this the structural and behavioural as parts of the system are represented as they are to be built. This view helps developers and engineers understand how the system will be implemented in practice, facilitating design decisions, coding, and testing. It is essential for ensuring that the final system aligns with the intended architecture and functional requirements.

- **Environmental Model View**

In this the structural and behavioural aspects of the environment in which the system is to be implemented are represented. It models external systems, hardware, network infrastructure, and user interactions that may affect the system's operation.

Class Diagram:

The class diagram is the main building block of object oriented modeling. It is used both for general conceptual modeling of the systematic of the application, and for detailed modeling translating the models into programming code. Class diagrams can also be used for data modeling. The classes in a class diagram represent both the main objects, interactions in the application and the classes to be programmed. In the diagram, classes are represented with boxes which contain three parts:

- The upper part holds the name of the class
- The middle part contains the attributes of the class
- The bottom part gives the methods or operations the class can take or undertake.

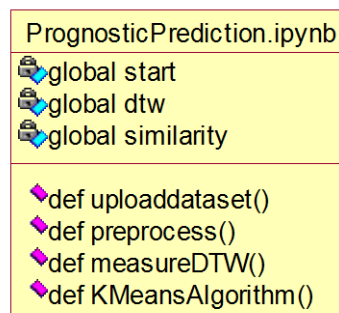


Fig 5.1.1 Class Diagram

Fig 5.1.1 The diagram shows the structure of the Prognostic Prediction module, including global variables and functions for data loading, preprocessing, similarity measurement (DTW), and clustering using K-Means.

Use case Diagram:

A use case diagram at its simplest is a representation of a user's interaction with the system and depicting the specifications of a use case. This diagram helps capture the functional requirements by showing the relationship between users and the system's functionalities. Use case diagrams are essential for visualizing system boundaries, understanding user goals, and facilitating communication among stakeholders during the design and development phases. Use case diagrams are widely used in software engineering to model the behavior of a system from an end-user perspective. They help identify the key functionalities the system must support and clarify the interactions between users and system components.

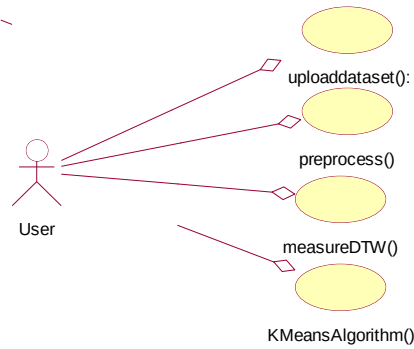


Fig 5.1.2 Use case diagram

Fig 5.1.2 The below diagram represents a use case diagram where a user (actor) interacts with multiple system functions, showing different operations that the user can perform within the system.

Sequence diagram:

A sequence diagram is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the Logical View of the system under development. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

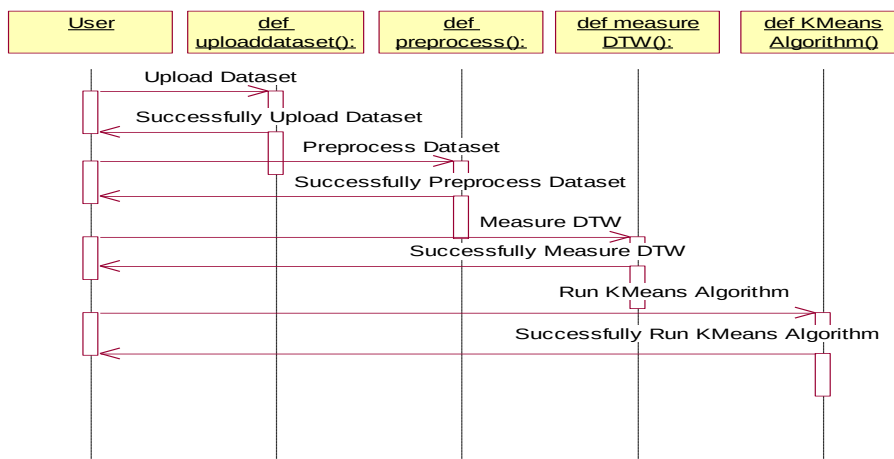


Fig 5.1.3 Sequence diagram

Fig 5.1.3 The diagram shows a simple workflow: the user uploads data, the system preprocesses it, calculates DTW distance, and then applies the K-Means algorithm. Each step is completed successfully in sequence.

Collaboration diagram:

A collaboration diagram describes interactions among objects in terms of sequenced messages. Collaboration diagrams represent a combination of information taken from class, sequence, and use case diagrams describing both the static structure and dynamic behaviour of a system.

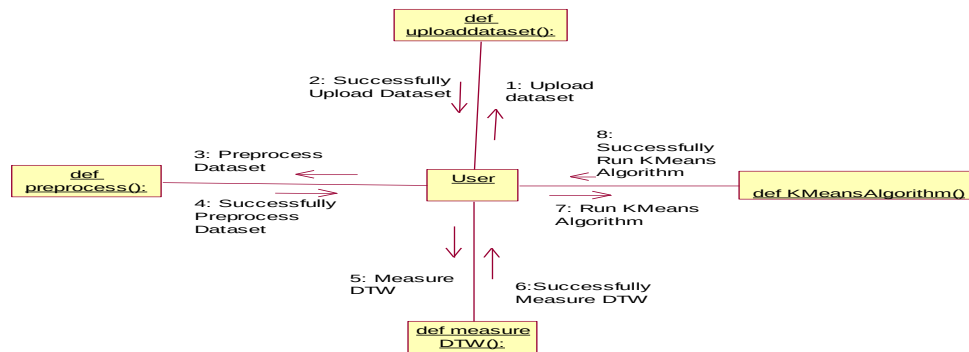


Fig 5.1.4 Collaboration diagram

Fig 5.1.4 This shows the workflow where the user uploads a dataset, the system preprocesses the data, calculates DTW, and applies the K-Means algorithm successfully.

Component Diagram:

In the Unified Modelling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems. This illustrates the service consumer - service provider relationship between the two components.

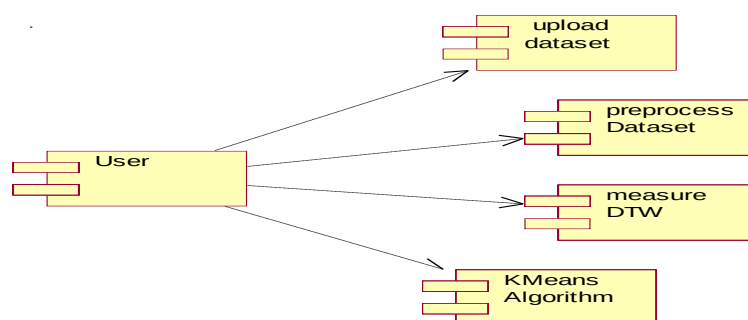


Fig 5.1.5 Component diagram

Fig 5.1.5 The diagram represents a component diagram where the user interacts with different system components such as uploading data, preprocessing, measuring DTW, and applying the K-Means algorithm.

Deployment Diagram:

A deployment diagram in the Unified Modeling Language models the *physical* deployment of artifacts on nodes. To describe a web site, for example, a deployment diagram would show what hardware components ("nodes") exist (e.g., a web server, an application server, and a database server), what software components ("artifacts") run on each node (e.g., web application, database), and how the different pieces are connected (e.g. JDBC, REST, RMI). The nodes appear as boxes, and the artifacts allocated to each node appear as rectangles within the boxes. Nodes may have sub nodes, which appear as nested boxes. A single node in a deployment diagram may conceptually represent multiple physical nodes, such as a cluster of database servers.

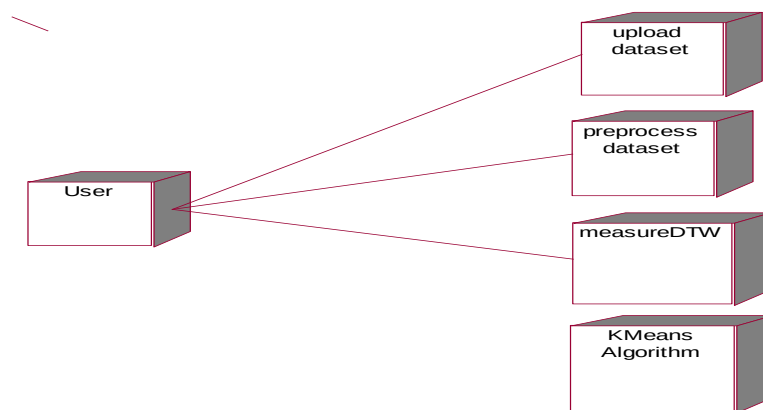


Fig 5.1.6 Deployment diagram

Fig 5.1.6 A central module performing operations by distributing tasks to multiple components for processing.

Activity Diagram:

Activity diagram is another important diagram in UML to describe dynamic aspects of the system. It is basically a flow chart to represent the flow from one activity to another activity. The activity can be described as an operation of the system. So the control flow is drawn from one operation to another. Activity diagrams are useful for modeling workflows, business processes, and detailed system behavior. They can represent sequential flows, decision points with branching paths, and concurrent activities happening in parallel. This makes them valuable tools for visualizing complex processes, improving understanding, and facilitating communication among stakeholders during system design and development.

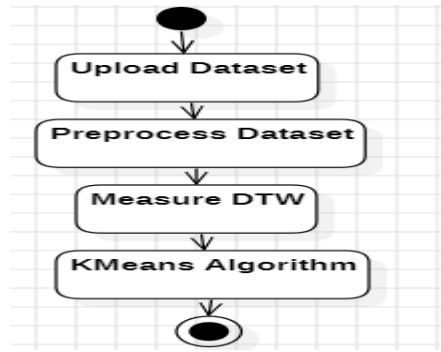


Fig 5.1.7 Activity diagram

Fig 5.1.7 Data preprocessing, DTW similarity measurement, and K-Means clustering.

5.2 DATA FLOW DIAGRAM

Data flow diagrams illustrate how data is processed by a system in terms of inputs and outputs. Data flow diagrams can be used to provide a clear representation of any business function. The technique starts with an overall picture of the business and continues by analyzing each of the functional areas of interest. This analysis can be carried out in precisely the level of detail required. The technique exploits a method called top-down expansion to conduct the analysis in a targeted way.

As the name suggests, Data Flow Diagram (DFD) is an illustration that explicates the passage of information in a process. A DFD can be easily drawn using simple symbols. Additionally, complicated processes can be easily automated by creating DFDs using easy-to-use, free downloadable diagramming tools. A DFD is a model for constructing and analyzing information processes. DFD illustrates the flow of information in a process depending upon the inputs and outputs. A DFD can also be referred to as a Process Model. A DFD demonstrates business or technical process with the support of the outside data saved, plus the data flowing from the process to another and the end results.

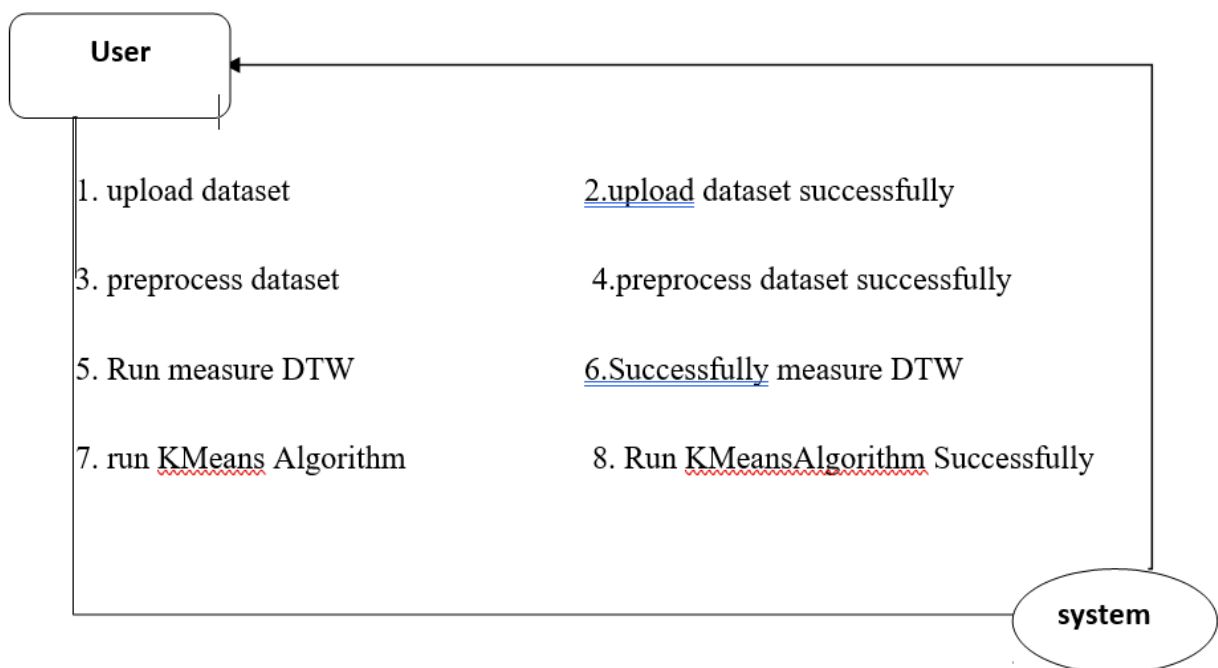


Fig 5.2.1 Data flow diagram

Fig 5.2.1 This diagram describes the process where the user uploads a dataset, the system preprocesses it, calculates DTW, and finally runs the K-Means algorithm, with each step completed successfully.

5.3 SYSTEM ARCHITECTURE

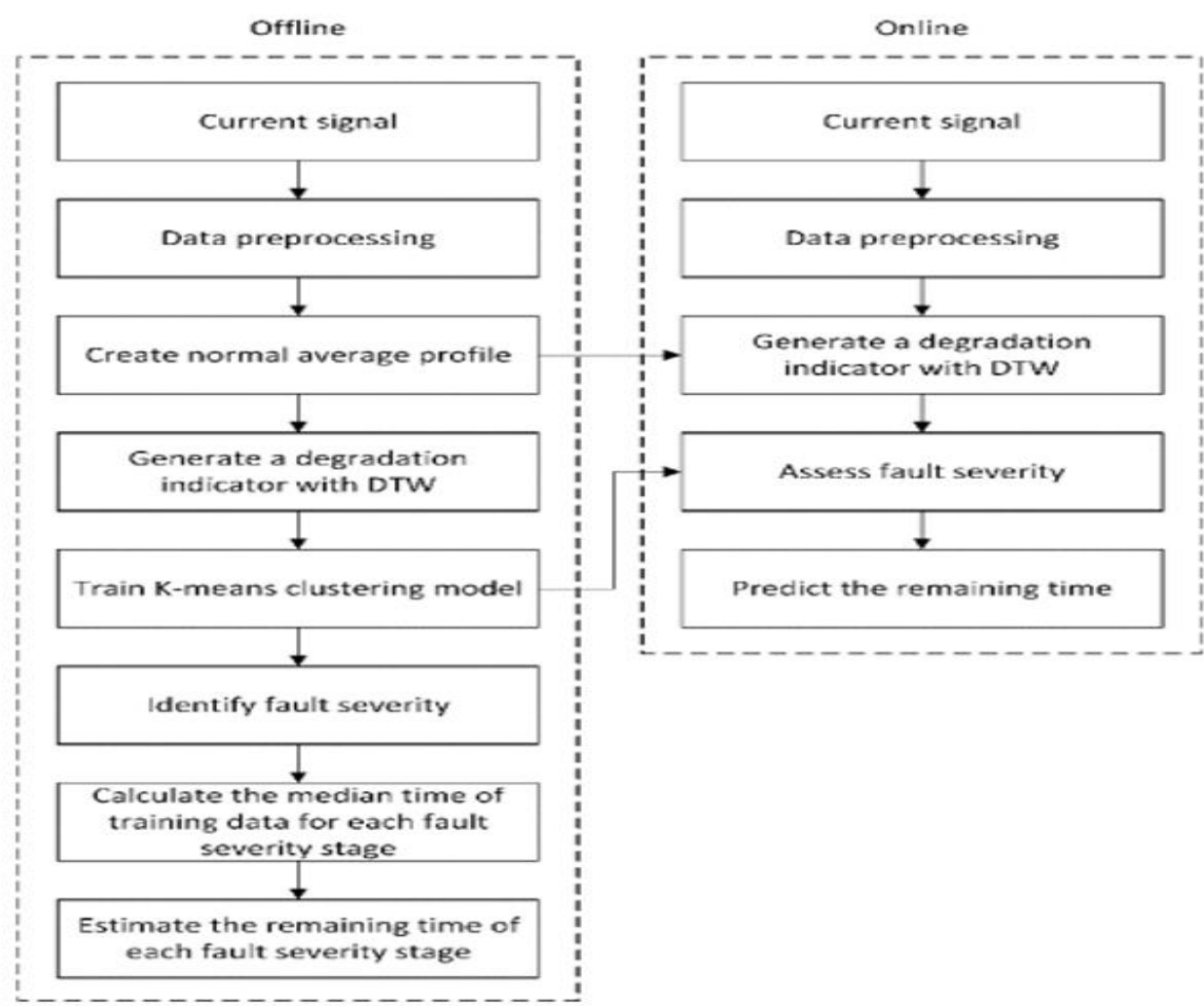


Fig 5.3.1 System architecture

Fig 5.3.1 This FlowChart represents offline and online processes for prognostic health management, where offline data is used to train models and identify fault patterns, while online data is analyzed in real time to assess fault severity and predict remaining useful life.

6. CODING AND IMPLEMENTATION

6.1 METHODOLOGY

PYTHON

* One of the most popular languages is Python. Guido van Rossum released this language in 1991. Python is available on the Mac, Windows, and Raspberry Pi operating systems. The syntax of Python is simple and identical to that of English. When compared to Python, it was seen that the other language requires a few extra lines.

*It is an interpreter-based language because code may be run line by line after it has been written. This implies that rapid prototyping is possible across all platforms. Python is a big language with a free, binary-distributed interpreter standard library.

* It is inferior to maintenance that is conducted and is straightforward to learn. It is an object-oriented, interpreted programming language. It supports several different programming paradigms in addition to object-oriented programming, including functional and procedural programming.

* It supports several different programming paradigms in addition to object-oriented programming, including practical and procedural programming. Python is mighty while maintaining a relatively straightforward syntax. Classes, highly dynamic data types, modules, and exceptions are covered. Python can also be utilised by programmes that require programmable interfaces as an external language.

Here are some key features and characteristics of Python:

- **Readability:** Python emphasizes code readability with its clean and intuitive syntax. It uses indentation and whitespace to structure code blocks, making it easy to understand and maintain.
- **Easy to Learn:** Python's simplicity and readability make it an excellent choice for beginners. Its straightforward syntax and extensive documentation make it accessible for newcomers to programming.
- **Interpreted Language:** Python is an interpreted language, meaning that it doesn't need to be compiled before running. The Python interpreter reads and executes the code directly, making the development process faster and more interactive.
- **Cross-platform Compatibility:** Python is available for major operating systems like Windows, macOS, and Linux. This cross-platform compatibility allows developers to write code once and run it on different platforms without modifications.

- **Large Standard Library:** Python comes with a vast standard library that provides ready-to-use modules and functions for various tasks. It covers areas such as file I/O, networking, regular expressions, databases, and more, saving developers time and effort.
- **Extensible and Modular:** Python supports modular programming, enabling developers to organize code into reusable modules and packages. Additionally, Python allows integrating modules written in other languages, such as C or C++, providing flexibility and performance optimizations.
- **Wide Range of Libraries and Frameworks:** Python has a vibrant ecosystem with numerous third-party libraries and frameworks. These libraries, such as NumPy, pandas, TensorFlow, and Django, extend Python's capabilities for specific domains, making it a powerful tool for diverse applications.
- **Object-Oriented:** Python supports object-oriented programming (OOP) principles, allowing developers to create and work with classes and objects. OOP provides a structured approach to code organization, promoting code reuse and modularity.
- **Dynamic Typing:** Python is dynamically typed, meaning variable types are determined at runtime. Developers do not need to declare variable types explicitly, which enhances flexibility and simplifies code writing.

➤ **Installation**

To install Python on your computer, follow these basic steps:

- **Step 1:** Visit the Python website Go to the official Python website at <https://www.python.org/>.
- **Step 2:** Select the operating system Choose the appropriate installer for your operating system. Python supports Windows, macOS, and various Linux distributions. Make sure to select the correct version that matches your operating system.
- **Step 3:** Check which version of Python is installed; if the 3.7.0 version is not there, uninstall it through the control panel and
- **Step 4:** Install Python 3.7.0 using Cmd.
- **Step 5:** Install the all libraries that required to run the project
- **Step 6:** Run

➤ **Python Features:**

- **Easy:** Because Python is a more accessible and straightforward language, Python programming is easier to learn.

- **Interpreted language:** Python is an interpreted language, therefore it can be used to examine the code line by line and provide results.
- **Open Source:** Python is a free online programming language since it is open-source.
- **Portable:** Python is portable because the same code may be used on several computer standard
- **libraries:** Python offers a sizable library that we may utilize to create applications quickly.
- **GUI:** It stands for GUI (Graphical User Interface)
- **Dynamical typed:** Python is a dynamically typed language, therefore the type of the value will be determined at runtime.

➤ **Python GUI (Tkinter)**

- Python provides a wide range of options for GUI development (Graphical User Interfaces).
- Tkinter, the most widely used GUI technique, is used for all of them.
- The Tk GUI toolkit offered by Python is used with the conventional Python interface.
- Tkinter is the easiest and quickest way to write Python GUI programs.
- Using Tkinter, creating a GUI is simple.
- A part of Python's built-in library is Tkinter. The GUI programs were created.
- Python and Tkinter together give a straightforward and quick way. The Tk GUI toolkit's object-oriented user interface is called Tkinter.

Making a GUI application is easy using Tkinter. Following are the steps:

- 1) Install the Tkinter module in place.
- 2) The GUI application Makes the primary window
- 3) Include one or more of the widgets mentioned above in the GUI application.
- 4) Set up the main event loop such that it reacts to each user-initiated event.

Although Tkinter is the only GUI framework included in the Python standard library, Python includes a GUI framework. The default library for Python is called Tkinter. Tk is a scripting language often used in designing, testing, and developing GUIs. Tk is a free, open-source widget toolkit that may be used to build GUI applications in a wide range of computer languages.

➤ **Python IDLE**

- ❖ Python IDLE offers a full-fledged file editor, which gives you the ability to write and execute Python programs from within this program. The built-in file editor also includes several features, like code completion and automatic indentation, that will speed up your coding workflow.

- ❖ Guido Van Rossum named Python after the British comedy group Monty Python while the name IDLE was chosen to pay tribute to Eric Idle, who was one of the Monty Python's founding members. IDLE comes bundled with the default implementation of the Python language since the 01.5. 2b1 release
- ❖ IDLE is used to execute statements similar to Python Shell. IDLE is used to create, modify, and execute Python code. IDLE provides a fully-featured text editor to write Python scripts and provides features like syntax highlighting, auto-completion, and smart indent.
- ❖ IDLE has two modes: interactive and script. We wrote our first program, “Hello, World!” in interactive mode. Interactive mode immediately returns the results of commands you enter into the shell. In script mode, you will write a script and then run it.
- ❖ The IDE Python IDLE is a good place to start as it helps you become familiar with the way Python works and understand its syntax. This IDE is good to start programming in Python due to its great debugger, but once you are fluent and start developing projects it is necessary to jump to another, more complete IDE.
- ❖ Python IDLE (Integrated Development and Learning Environment) is an interactive development environment included with the Python programming language. It provides a convenient way to write, execute, and debug Python code.

When you install Python, IDLE is typically installed along with it. To open IDLE, you can follow these steps:

- Open the command prompt (Windows) or terminal (macOS/Linux).
- Type "idle" and press Enter. Alternatively, you can specify the version with "idle3" or "idle2" for Python 3 or Python 2, respectively.
- Once IDLE is launched, you will see the Python shell, which is an interactive environment where you can type and execute Python code directly.

Here are some features and functionalities provided by Python IDLE:

- Editor: IDLE includes a text editor where you can write your Python code. It offers syntax highlighting, automatic indentation, and code completion to enhance your coding experience.
- Interactive Shell: The Python shell in IDLE allows you to execute Python code interactively. You can type commands, statements, or function calls directly in the shell, and Python will execute them immediately.

- **Debugging:** IDLE provides basic debugging capabilities to help you find and fix errors in your code. You can set breakpoints, step through code, inspect variables, and track the program's execution.
- **Python Help:** IDLE provides access to the Python documentation and built-in help. You can access the help menu to find information about Python modules, functions, classes, and more.
- **Script Execution:** In addition to the interactive shell, IDLE allows you to run Python scripts stored in files. You can write your code in the editor and execute it as a script to see the output or interact with the program.
- **Customization:** IDLE can be customized to suit your preferences. You can modify settings related to syntax highlighting, indentation, fonts, and more.
- **Python IDLE serves as a beginner-friendly development environment and learning tool.** It is suitable for writing small scripts, testing code snippets, experimenting with Python features, and learning the language's basics. However, for more advanced development projects, you may consider using other code editors or integrated development environments (IDEs) that provide additional features and better project management capabilities.

➤ **Libraries**

- In Python, libraries (also referred to as modules or packages) are collections of pre-written code that provide additional functionality and tools to extend the capabilities of the Python language. Libraries contain reusable code that developers can leverage to perform specific tasks without having to write everything from scratch.
- Python libraries are designed to solve common problems, such as handling data, performing mathematical operations, interacting with databases, working with files, implementing networking protocols, creating graphical user interfaces (GUIs), and much more. They provide ready-to-use functions, classes, and methods that simplify complex operations and save development time.

Libraries in Python offer various advantages:

- **Code Reusability:**
- **Efficiency:**
- **Collaboration**
- **Domain-Specific Functionality**
- **To use a Python library, you need to install it first.**

There are some libraries following:

➤ **Pandas:**

Pandas are a Python computer language library for data analysis and manipulation. It offers a specific operation and data format for handling time series and numerical tables. It differs significantly from the release3-clause of the BSD license. It is a well-liked open-source of opinion that is utilized in machine learning and data analysis. Pandas are a Python package providing fast, flexible, and expressive data structures designed to make working with “relational” or “labeled” data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, real-world data analysis in Python. Pandas are a Python library used for working with data sets.

- It has functions for analysing, cleaning, exploring, and manipulating data.
- The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.
- Pandas allow us to analyse big data and make conclusions based on statistical theories.
- Pandas can clean messy data sets, and make them readable and relevant.

Relevant data is very important in data science. Pandas are a Python library for data analysis. Started by Wes McKinney in 2008 out of a need for a powerful and flexible quantitative analysis tool, pandas have grown into one of the most popular Python libraries. It has an extremely active community of contributors. The name is derived from the term "panel data", an econometrics term for data sets that include observations over multiple time periods for the same individuals. Its name is a play on the phrase "Python data analysis" itself.

➤ **NumPy:**

The NumPy Python library for multi-dimensional, big-scale matrices adds a huge number of high-level mathematical functions. It is possible to modify NumPy by utilizing a Python library. Along with line, algebra, and the Fourier transform operations, it also contains several matrices-related functions. NumPy can be used to perform a wide variety of mathematical operations on arrays. It adds powerful data structures to Python that guarantee efficient calculations with arrays and matrices and it supplies an enormous library of high-level mathematical functions that operate on these arrays and matrices.

- NumPy is a Python library used for working with arrays.
- It also has functions for working in domain of linear algebra, Fourier transform, and matrices.

- NumPy was created in 2005 by Travis Oliphant. It is an open source project and you can use it freely.
- NumPy stands for Numerical Python.
- In Python we have lists that serve the purpose of arrays, but they are slow to process.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called nd array, it provides a lot of supporting functions that make working with nd array very easy.
- Arrays are very frequently used in data science, where speed and resources are very important.

➤ **Matplotlib:**

It is a multi-platform, array-based data visualization framework built to interact with the whole SciPy stack. MATLAB is proposed as an open-source alternative. Matplotlib is a Python extension and a cross-platform toolkit for graphical plotting and visualization. Matplotlib is a popular Python library for creating static, animated, and interactive visualizations. It provides a flexible and comprehensive set of tools for generating plots, charts, histograms, scatter plots, and more. Matplotlib is widely used in various fields, including data analysis, scientific research, and data visualization.

Here are some key features and functionalities of the Matplotlib library:

- Plotting Functions
- Customization Options
- Multiple Interfaces
- Integration with NumPy and pandas
- Subplots and Figures:
- Saving and Exporting

➤ **Scikit-learn:**

The most stable and practical machine learning library for Python is scikit-learn. Regression, dimensionality reduction, classification, and clustering are just a few of the helpful tools it provides through the Python interface for statistical modeling and machine learning. It is an essential part of the Python machine learning toolbox used by JP Morgan. It is frequently used in various machine learning applications, including classification and predictive analysis. Scikit-learn (also referred to as sklearn) is a widely used open-source machine learning library for Python. It provides a comprehensive set of tools and algorithms for various machine learning tasks, including classification, regression, clustering, dimensionality reduction, model selection, and pre-processing.

Here are some key features and functionalities of the Scikit-learn library:

- Easy-to-Use Interface:
- Broad Range of Algorithms:
- Data Pre-processing and Feature Engineering:
- Model Evaluation and Validation:
- Integration with NumPy and pandas:
- Robust Documentation and Community Support:

➤ **Keras:**

* Google's Keras is a cutting-edge deep learning API for creating neural networks. It is created in Python and is designed to simplify the development of neural networks. Additionally, it enables the use of various neural networks for computation. Deep learning models are developed and tested using the free and open-source Python software known as Keras. Keras is a high-level deep learning library for Python. It is designed to provide a user-friendly and intuitive interface for building and training deep learning models. Keras acts as a front-end API, allowing developers to define and configure neural networks while leveraging the computational backend engines, such as Tensor Flow or Theano.

Here are some key features and functionalities of the Keras library:

- User-Friendly API
- Multi-backend Support
- Wide Range of Neural Network Architectures
- Pre-trained Models and Transfer Learning:
- Easy Model Training and Evaluation:
- GPU Support:

➤ **h5py:**

The h5py Python module offers an interface for the binary HDF5 data format. Thanks to p5py, the top can quickly halt the vast amount of numerical data and alter it using the NumPy library. It employs common syntax for Python, NumPy, and dictionary arrays. h5py is a Python library that provides a simple and efficient interface for working with datasets and files in the Hierarchical Data Format 5 (HDF5) format. HDF5 is a versatile data format commonly used for storing and managing large volumes of numerical data.

Here are some key features and functionalities of the h5py library:

- HDF5 File Access

- Dataset Handling:
- Group Organization:
- Attributes:
- Compatibility with NumPy
- Performance

➤ **Tensor flow**

TensorFlow is a Python library for fast numerical computing created and released by Google. It is a foundation library that can be used to create Deep Learning models directly or by using wrapper libraries that simplify the process built on top of TensorFlow. TensorFlow is an end-to-end open source platform for machine learning. TensorFlow is a rich system for managing all aspects of a machine learning system; however, this class focuses on using a particular TensorFlow API to develop and train machine learning models. TensorFlow is a popular open-source library for machine learning and deep learning. It provides a comprehensive set of tools, APIs, and computational resources for building and training various types of machine learning models, especially neural networks.

Here are some key features and functionalities of TensorFlow:

- Neural Network Framework:
- Computational Graphs
- Automatic Differentiation
- GPU and TPU Support
- Distributed Computing
- Deployment Capabilities

➤ **Tkinter**

Tkinter is an acronym for "Tk interface". Tk was developed as a GUI extension for the Tcl scripting language by John Ousterhout. The first release was in 1991. Tkinter is the de facto way in Python to create Graphical User interfaces (GUIs) and is included in all standard Python Distributions. In fact, it's the only framework built into the Python standard library. Tkinter is a standard Python library used for creating graphical user interfaces (GUIs). It provides a set of modules and classes that allow you to develop interactive and visually appealing desktop applications.

Here are some key features and functionalities of Tkinter:

- Cross-Platform Compatibility
- Simple and Easy-to-Use
- Widgets and Layout Management
- Event-Driven Programming
- Customization and Styling.

➤ **NLTK**

NLTK is a toolkit build for working with NLP in Python. It provides us various text processing libraries with a lot of test datasets. A variety of tasks can be performed using NLTK such as tokenizing, parse tree visualization, etc NLTK (Natural Language Toolkit) is the go-to API for NLP (Natural Language Processing) with Python. It is a really powerful tool to pre-process text data for further analysis like with ML models for instance. It helps convert text into numbers, which the model can then easily work with. NLTK (Natural Language Toolkit) is a Python library widely used for working with human language data and implementing natural language processing (NLP) tasks. It provides a set of tools, corpora, and resources for tasks such as tokenization, stemming, tagging, parsing, sentiment analysis, and more.

Here are some key features and functionalities of NLTK:

- Text Processing
- Part-of-Speech Tagging
- Named Entity Recognition
- Chunking and Parsing
- Sentiment Analysis:
- WordNet Integration:

➤ **Scipy**

SciPy is a collection of mathematical algorithms and convenience functions built on the NumPy extension of Python. It adds significant power to the interactive Python session by providing the user with high-level commands and classes for manipulating and visualizing data. SciPy is a powerful scientific computing library for Python that provides a wide range of mathematical algorithms and functions. It builds upon NumPy, another fundamental library for numerical computing, and extends its capabilities by adding additional tools for scientific and technical computing tasks.

Here are some key features and functionalities of SciPy:

- Numerical Integration:

- Optimization and Root Finding
- Linear Algebra
- Signal and Image Processing
- Statistics

➤ **K-Means Algorithm:**

- Unsupervised Machine Learning is the process of teaching a computer to use unlabeled, unclassified data and enabling the algorithm to operate on that data without supervision. Without any previous data training, the machine's job in this case is to organize unsorted data according to parallels, patterns, and variations.
- K means clustering, assigns data points to one of the K clusters depending on their distance from the center of the clusters. It starts by randomly assigning the clusters centroid in the space. Then each data point assign to one of the cluster based on its distance from centroid of the cluster. After assigning each point to one of the cluster, new cluster centroids are assigned. This process runs iteratively until it finds good cluster. In the analysis we assume that number of cluster is given in advanced and we have to put points in one of the group.
- In some cases, K is not clearly defined, and we have to think about the optimal number of K. K Means clustering performs best data is well separated. When data points overlapped this clustering is not suitable. K Means is faster as compare to other clustering technique. It provides strong coupling between the data points. K Means cluster do not provide clear information regarding the quality of clusters. Different initial assignment of cluster centroid may lead to different clusters. Also, K Means algorithm is sensitive to noise. It may have stuck in local minima.

What is the objective of k-means clustering?

The goal of clustering is to divide the population or set of data points into a number of groups so that the data points within each group are more comparable to one another and different from the data points within the other groups. It is essentially a grouping of things based on how similar and different they are to one another.

How k-means clustering works?

We are given a data set of items, with certain features, and values for these features (like a vector). The task is to categorize those items into groups. To achieve this, we will use the K-means algorithm, an unsupervised learning algorithm. 'K' in the name of the algorithm represents the number of groups/clusters we want to classify our items into.(It will help if you think of items as points in an n-

dimensional space). The algorithm will categorize the items into k groups or clusters of similarity. To calculate that similarity, we will use the Euclidean distance as a measurement.

The algorithm works as follows:

1. First, we randomly initialize k points, called means or cluster centroids.
2. We categorize each item to its closest mean, and we update the mean's coordinates, which are the averages of the items categorized in that cluster so far.
3. We repeat the process for a given number of iterations and at the end, we have our clusters.

The “points” mentioned above are called means because they are the mean values of the items categorized in them. To initialize these means, we have a lot of options. An intuitive method is to initialize the means at random items in the data set. Another method is to initialize the means at random values between the boundaries of the data set (if for a feature x , the items have values in $[0,3]$, we will initialize the means with values for x at $[0,3]$).

6.2 SAMPLE CODE:

Views.py

```
#importing require classes and packages

import pandas as pd

import numpy as np

import tslearn.metrics

from sklearn.cluster import KMeans

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

#function to calculate DTW from electric floor current values using time series from old to new values

def measureDTW(dataset):

    start = dataset[0] #get first value

    dtw = []

    similarity = tslearn.metrics.dtw(start, dataset[1]) # Get only the similarity score for DTW

    dtw.append(similarity)

    for i in range(1, len(dataset)): #now loop entire dataset to get DTW from all values
```

```

        similarity = tslearn.metrics.dtw(start, dataset[i]) # Get only the similarity score for DTW
        start = dataset[i]

        dtw.append(similarity)

    return np.asarray(dtw)

#read and display dataset values
dataset = pd.read_csv("Dataset/Train_Data_CSV.csv")
dataset.fillna(0, inplace = True)
dataset

#now calculate DTW similarity for each current value
dataset.drop(['Data_No','RUL', 'Bias_type'], axis = 1,inplace=True)
dtw_similarity = measureDTW(dataset.values)
dataset['dtw'] = dtw_similarity
print("DTW Similarity")
print(dtw_similarity )

#now train KMeans algorithm on dtw values to generate 4 level clusters
dtw_similarity_values = dtw_similarity.reshape(-1, 1)
kmeans = KMeans(n_clusters=4)#create 4 clusters
kmeans.fit(dtw_similarity_values)#training kmeans using dtw values
labels = kmeans.labels_
dataset['clusters'] = labels
print("KMeans Clustering Completed")

#plot 4 levels clusters
cls1 = dataset.loc[dataset['clusters'] == 0]
cls2 = dataset.loc[dataset['clusters'] == 1]
cls3 = dataset.loc[dataset['clusters'] == 2]
cls4 = dataset.loc[dataset['clusters'] == 3]
plt.figure(figsize=(10,6))

```



```

plt.grid(True)

plt.xlabel('Epoch')

plt.ylabel('Accuracy/Loss')

plt.plot(cls1['Current'].ravel()[0:200], color = 'green')

plt.plot(cls2['Current'].ravel()[0:200], color = 'magenta')

plt.plot(cls3['Current'].ravel()[0:200], color = 'black')

plt.plot(cls4['Current'].ravel()[0:200], color = 'red')

plt.legend(['Level-1', 'Level-2','Level-3','Level-4'], loc='upper left')

plt.title('Current Signals of Severity Assessment Levels Graph')

plt.show()

#now plot clusters observation graph

plt.figure(figsize=(10,4), dpi=100)

plt.scatter(cls1['dtw'].ravel()[0:200] , cls1['Current'].ravel()[0:200], c='red')

plt.scatter(cls2['dtw'].ravel()[0:200] , cls2['Current'].ravel()[0:200], c='green')

plt.scatter(cls3['dtw'].ravel()[0:200] , cls3['Current'].ravel()[0:200], c='blue')

plt.scatter(cls4['dtw'].ravel()[0:200] , cls4['Current'].ravel()[0:200], c='yellow')

plt.legend(['Stage-1', 'Stage-2','Stage-3','Stage-4'], loc='upper left')

plt.show()

#now calculate lifetime using 10 clusters

lifetime = []

max_value=np.max([np.max(cls4['Current'].ravel()),np.max(cls3['Current'].ravel()),
np.max(cls2['Current'].ravel()),np.max(cls1['Current'].ravel())])

stage_1 = max_value - np.max(cls1['Current'].ravel())

stage_2 = max_value - np.max(cls2['Current'].ravel())

stage_3 = max_value - np.max(cls3['Current'].ravel())

stage_4 = max_value - np.max(cls4['Current'].ravel())

values = []

values.append(stage_1)

```

```

values.append(stage_2)
values.append(stage_3)
values.append(stage_4)
values.sort(reverse=True)

lifetime.append(['RUL', values[0], values[1], values[2], values[3]])

data = pd.DataFrame(lifetime, columns=['LifeTime', 'Stage1', 'Stage2', 'Stage3', 'Stage4'])

data

#performance of Kmeans clustering with and without tuning

kmeans_score = abs(kmeans.score(dtw_similarity_values))

print("Propose KMEANS Score : "+str(kmeans_score))

itr = [50, 100, 150, 200, 300, 500]

extension_score = 0

for i in range(len(itr)):

    #create 4 clusters with given iteration optimization

    #training Kmeans with optimized parameters till score get better

    ext_kmeans = KMeans(n_clusters=4, algorithm="auto", max_iter=itr[i], tol=0.00000001)

    ext_kmeans.fit(dtw_similarity_values)#training kmeans using dtw values

    score = abs(ext_kmeans.score(dtw_similarity_values))

    if score < kmeans_score:

        extension_score = score

print("Extension Tuned Kmeans Score : "+str(extension_score))

#comparison between propose and extension algorithms

height = [kmeans_score, extension_score]

bars = ['Propose Kmeans Score', 'Tuned KMeans Score']

y_pos = np.arange(len(bars))

plt.bar(y_pos, height)

plt.xticks(y_pos, bars)

```

```
plt.xlabel("Algorithm Names")  
plt.ylabel("Cluster Score")  
plt.title("Propose & Extension Tuned Kmeans Score Graph")  
plt.show()
```

7. TESTING

7.1 INTRODUCTION TO TESTING

Implementation and Testing:

Implementation is one of the most important tasks in project is the phase in which one has to be cautious because all the efforts undertaken during the project will be very interactive. Implementation is the most crucial stage in achieving successful system and giving the users confidence that the new system is workable and effective. Each program is tested individually at the time of development using the sample data and has verified that these programs link together in the way specified in the program specification. The computer system and its environment are tested to the satisfaction of the user.

Implementation

The implementation phase is less creative than system design. It is primarily concerned with user training, and file conversion. The system may be requiring extensive user training. The initial parameters of the system should be modified as a result of a programming. A simple operating procedure is provided so that the user can understand the different functions clearly and quickly. The different reports can be obtained either on the inkjet or dot matrix printer, which is available at the disposal of the user. The proposed system is very easy to implement. In general implementation is used to mean the process of converting a new or revised system design into an operational one.

Testing

Testing is the process where the test data is prepared and is used for testing the modules individually and later the validation given for the fields. Then the system testing takes place which makes sure that all components of the system property functions as a unit. The test data should be chosen such that it passed through all possible condition. Actually testing is the state of implementation which aimed at ensuring that the system works accurately and efficiently before the actual operation commence.

7.2 SYSTEM TESTING

Testing has become an integral part of any system or project especially in the field of information technology. The importance of testing is a method of justifying, if one is ready to move further, be

it to be check if one is capable to with stand the rigors of a particular situation cannot be underplayed and that is why testing before development is so critical. When the software is developed before it is given to user to use the software must be tested whether it is solving the purpose for which it is developed. This testing involves various types through which one can ensure the software is reliable. The program was tested logically and pattern of execution of the program for a set of data are repeated.

7.3 MODULE TESTING

To locate errors, each module is tested individually. This enables us to detect error and correct it without affecting any other modules. Whenever the program is not satisfying the required function, it must be corrected to get the required result. Thus all the modules are individually tested from bottom up starting with the smallest and lowest modules and proceeding to the next level. Each module in the system is tested separately. For example the job classification module is tested separately. This module is tested with different job and its approximate execution time and the result of the test is compared with the results that are prepared manually. The comparison shows that the results proposed system works efficiently than the existing system. Each module in the system is tested separately. In this system the resource classification and job scheduling modules are tested separately and their corresponding results are obtained which reduces the process waiting time.

7.4 INTEGRATION TESTING

After the module testing, the integration testing is applied. When linking the modules there may be chance for errors to occur, these errors are corrected by using this testing. In this system all modules are connected and tested. The testing results are very correct. Thus the mapping of jobs with resources is done correctly by the system.

7.5 ACCEPTANCE TESTING

When that user find no major problems with its accuracy, the system passers through a final acceptance test. This test confirms that the system needs the original goals, objectives and requirements established during analysis without actual execution which elimination wastage of time and money acceptance tests on the shoulders of users and management, it is finally acceptable and ready for the operation.

Table 7.5 Acceptance Testing

Test Case Id	Test Case Name	Test Case Desc.	Test Steps			Test Case Status	Test Priority
			Step	Expected	Actual		
01	Upload Dataset	Test whether Dataset is uploaded or not into the system	If the Dataset may not uploaded	We cannot do further operations	Dataset uploaded we will do further operations	High	High
02	Run Measure DTW	Test whether Measure DTW Run Successfully or not	If the Measure DTW Algorithm may not Run Successfully	We cannot do further operations	we will do further operations	High	High
03	Preprocess & Normalized Dataset	Test whether the Pre-process & Normalized Dataset Successfully or not	If the Pre - process & Normalized Dataset may not Run Successfully	We cannot do further operations	we will do further operations	High	High
04	Train run KMeans Algorithm	Test whether KMeans Algorithm Run Successfully or not	If the KMeans Algorithm may not Run Successfully	We cannot do further operations	we will do further operations	High	High

Table 7.5 This table presents the acceptance testing of the system, verifying key functionalities such as dataset upload, preprocessing and normalization, DTW measurement, and K-Means algorithm execution. Each test case ensures that the respective operation runs successfully, enabling smooth continuation of further processes in the system.

8. OUTPUT SCREENS

To Run the application Click on “run.bat” file from the file location.

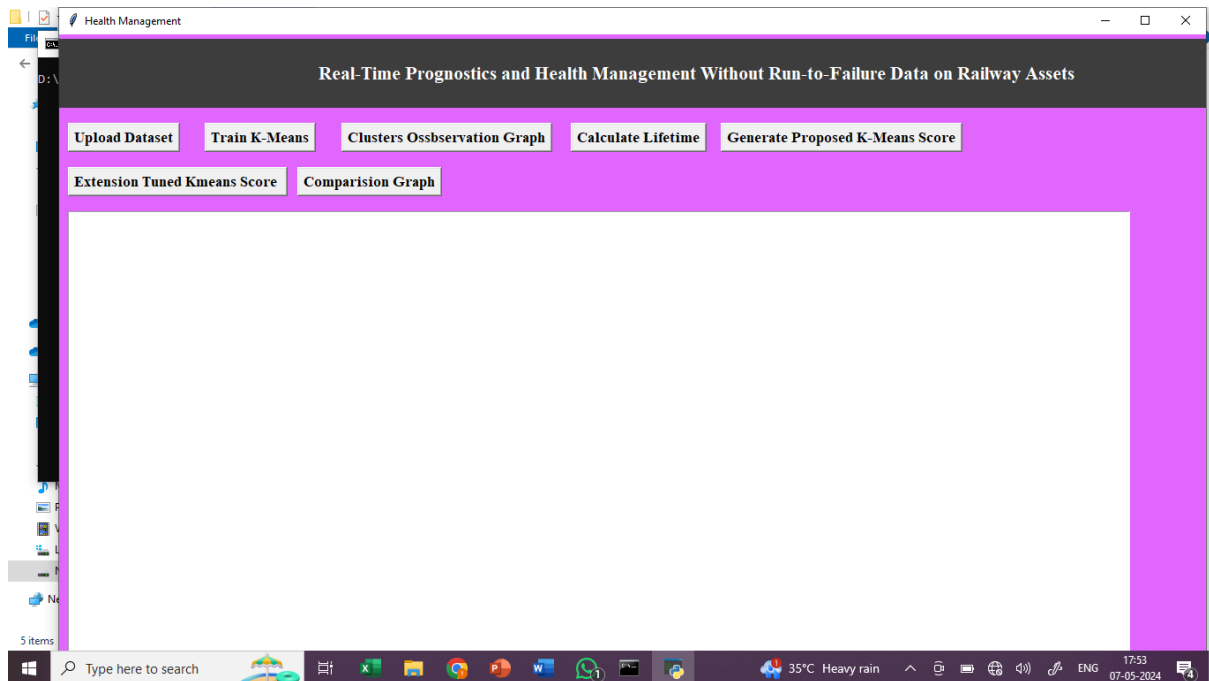


Fig 8.1 Tkinter Application Main Window

Fig 8.1 In the above screen we got Tkinter Output Window. Now Click On the “Upload Dataset” button to upload the dataset to the application.

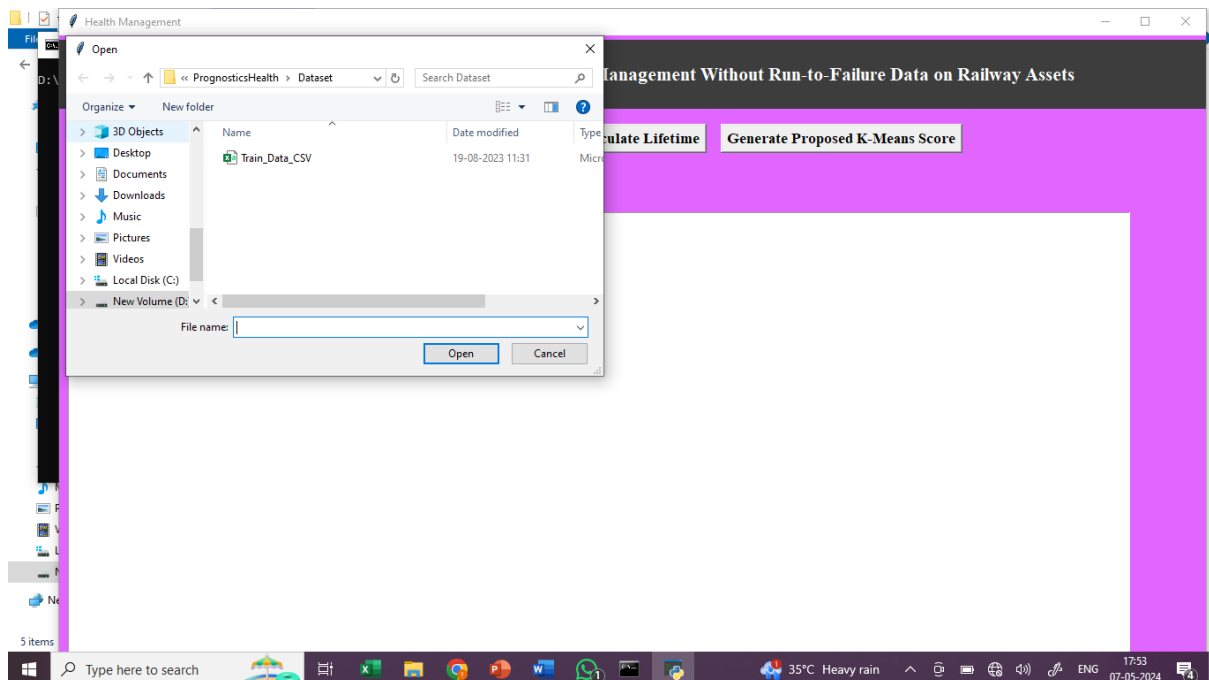


Fig 8.2 Uploading Dataset to the Application

Fig 8.2 In the above we can see the dataset uploading.

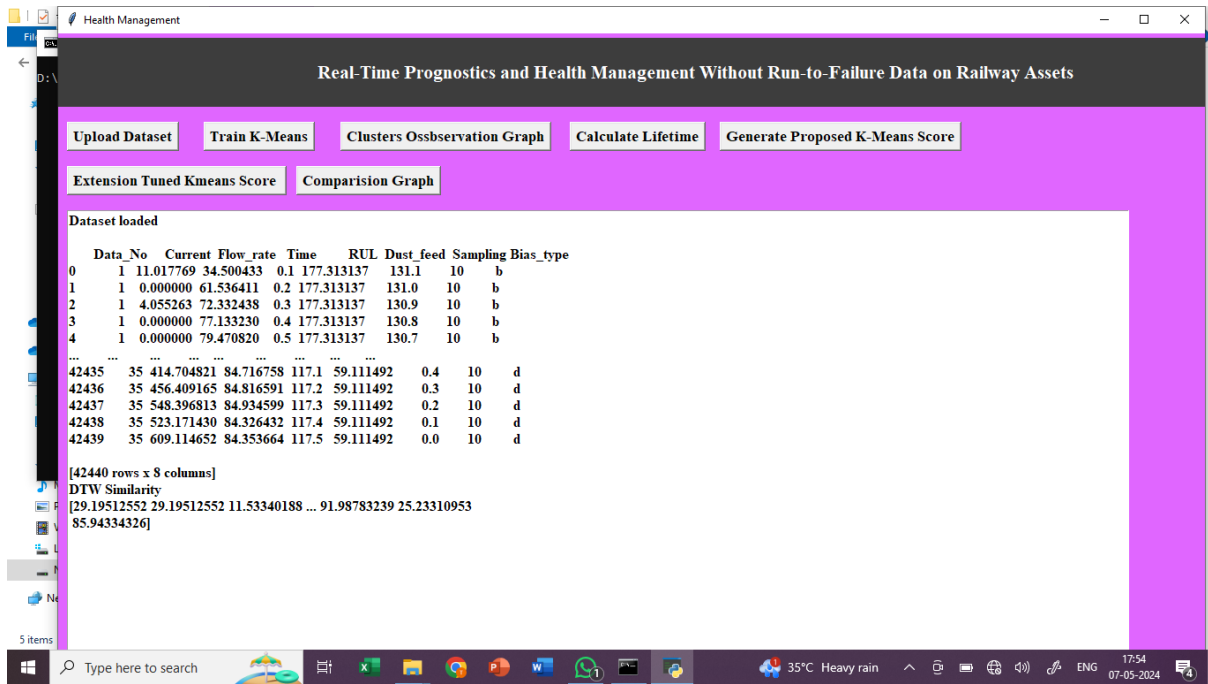


Fig 8.3 Loading and Displaying Dataset Values

Fig 8.3 In above screen loading and displaying dataset values.

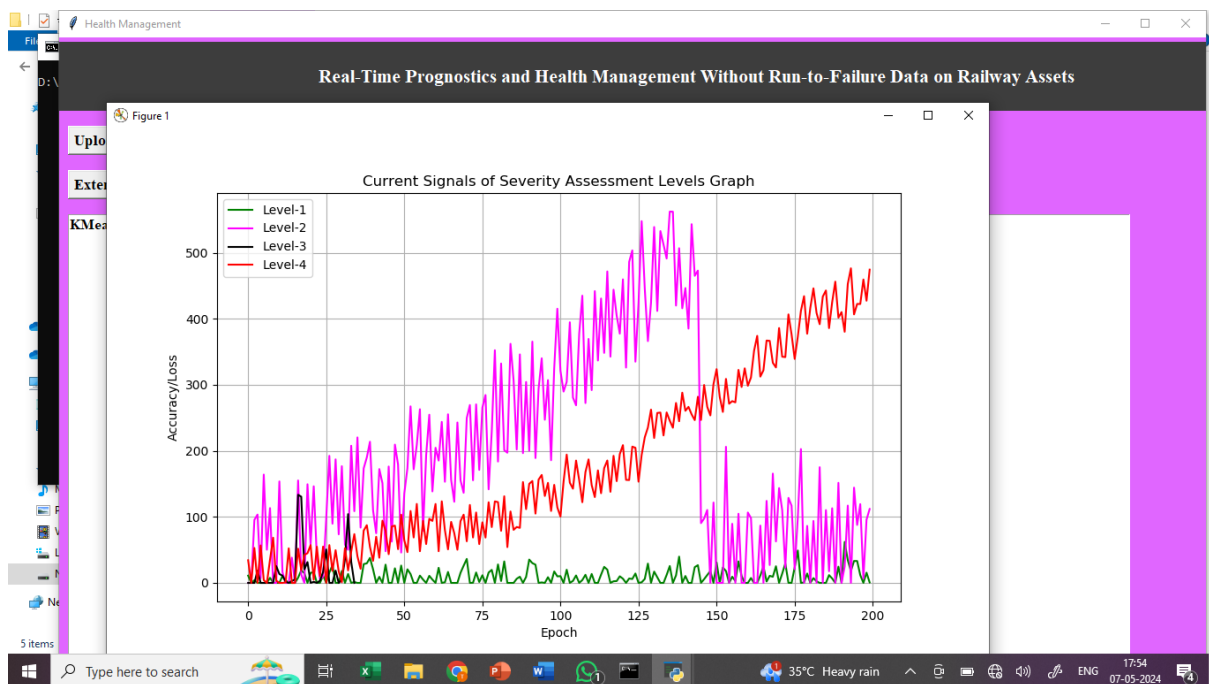


Fig 8.4 K-Means Clustering of Motor Current Values

Fig 8.4 Above is the 4 cluster graph where x-axis represents MOTOR current index and y-axis represents motor current values. Different line represents different values in cluster and in above graph we can see cluster in green colour line has normal values without any much gap. In black, magenta and red colour lines cluster we can see lines are having too much up and drops so it contains faulty values. So by using KMEANS we can see which cluster values are faulty.

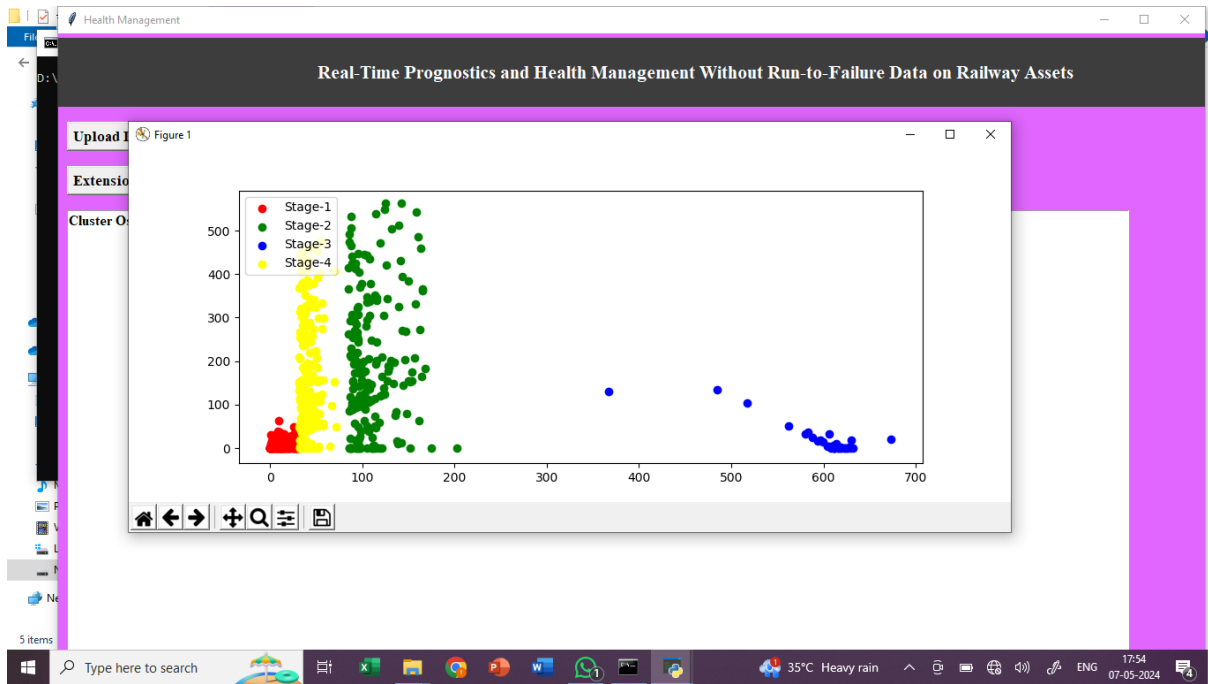


Fig 8.5 Stage-wise Lifetime Estimation from Clusters

Fig 8.5 In above screen each colour circle represents one stage and the ending and starting difference of each cluster values will be consider as Lifetime and after finding median and subtracting above cluster values we will get below lifetime for each stage.

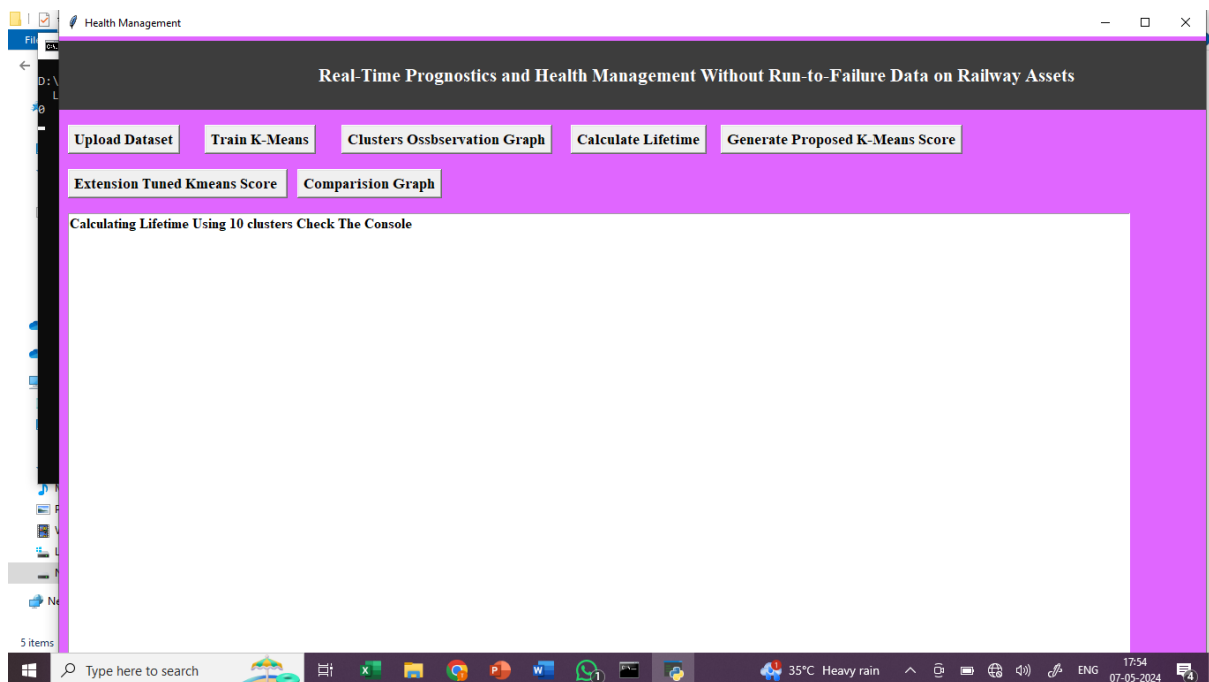


Fig 8.6 Calculating Lifetime Using 10 Clusters

Fig 8.6 In above screen click on 'Calculate Lifetime' then it finds the Calculating Lifetime Using 10 clusters

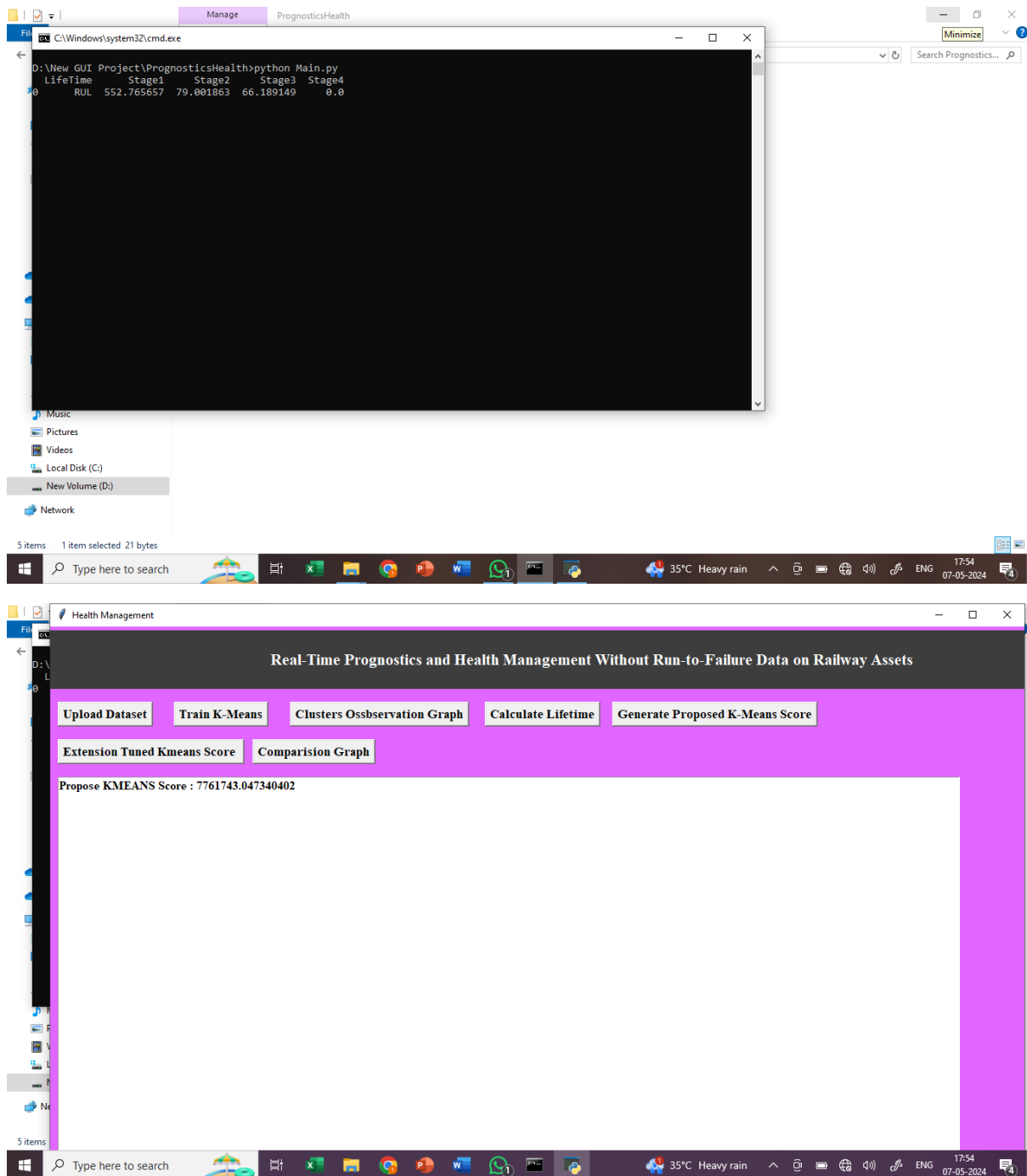


Fig 8.7 Proposed K-Means Score for Cluster Differences

Fig 8.7 In above screen in blue colour text we can see propose KMEANS score which will consider as difference between each cluster value

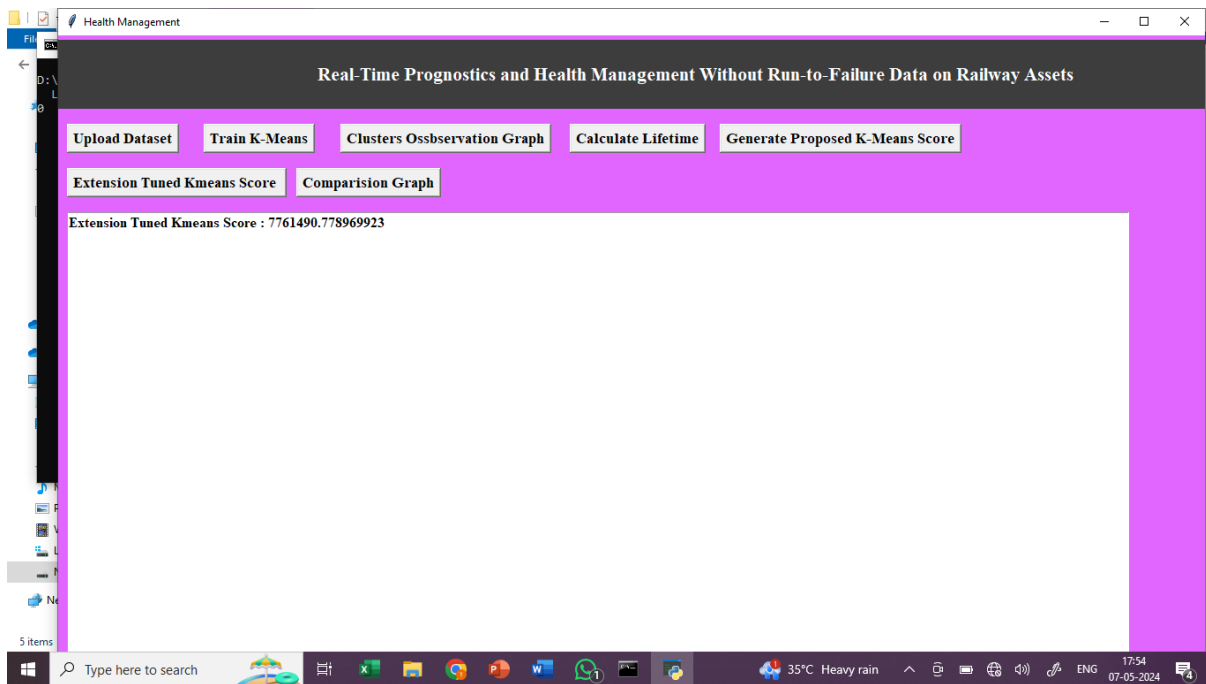


Fig 8.8 Optimizing K-Means through Iterative Tuning

Fig 8.8 In above screen we are tuning KMEANS with various iterations and then in last we got KMEANS with lesser score and this less score KMEANS object is the optimized KMEANS.

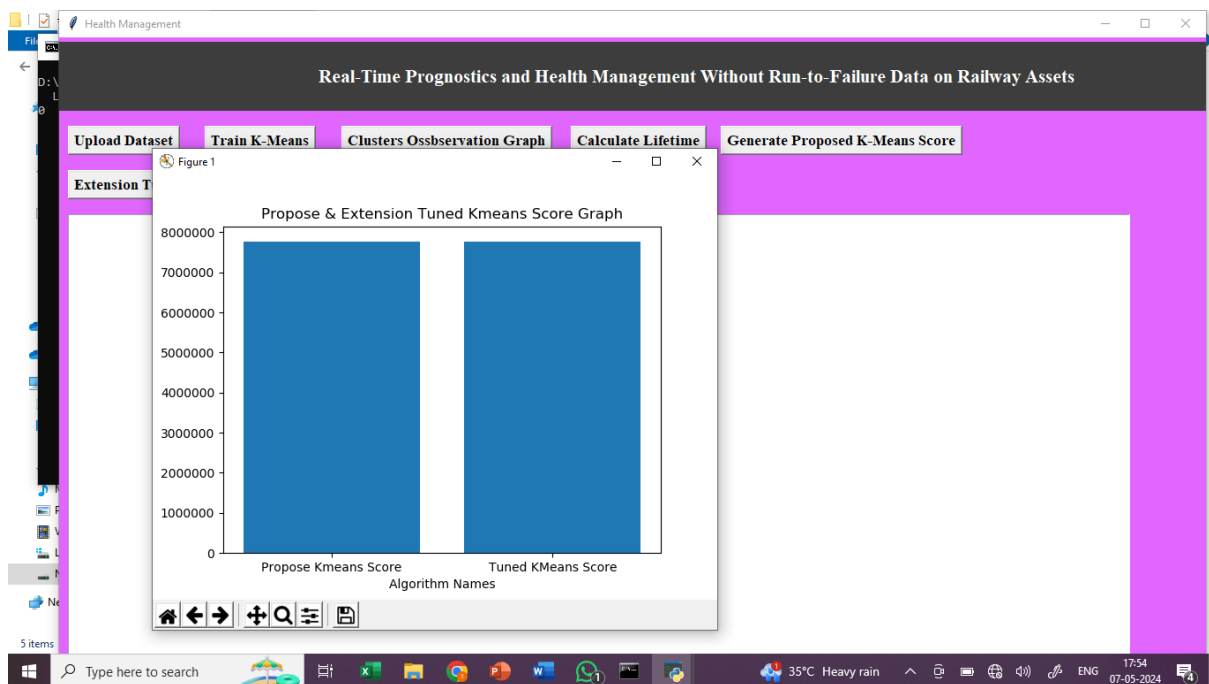


Fig 8.9 Comparison of Algorithm Scores (K-Means Optimized)

Fig 8.9 In above graph x-axis represents algorithm names and y-axis represents SCORE and in both algorithm extension KMEANS got less score

9. CONCLUSION

This novel prognosis method that does not require RTF data is an applicable and useful tool for real-world railway maintenance planning. The key idea is that fault severity assessments and approximate remaining time predictions are often all that is needed in order to plan maintenance. Therefore, this paper established a practical prognosis methodology that uses fault severity assessments and approximate remaining time predictions instead of RTF data to estimate accurate RUL. Firstly, a degradation indicator on railway door systems is generated using motor current signals based on the DTW method to compare the behaviour of typical normal and faulty door systems. Then the K-means algorithm is applied to assess fault severity, followed by the time estimation of each fault severity level. To be concise, this estimation allows the remaining time until reaching the critical fault severity level to be predicted without using RTF data. The proposed method offers significant advantages in terms of practical real-time prognostics and health management. Firstly, the proposed method can be applied to real-world railway maintenance planning as the remaining time can be predicted in real-time without using RTF data, which has previously been a great setback to building a prediction model. Secondly, the fault severity threshold can be updated by including additional operational data, enabling the remaining time prediction to be more reliable. The RUL can also be predicted using the proposed method once enough RTF data has been gathered during the operation. Additionally, the proposed method can also be applied to conventional railway assets and other EMAs as motor current signals are primarily available from the controller or motor drive without additional sensors, which is also beneficial for practical prognostic health management systems. Determining a confidence level in the prediction was out of this paper's scope, however future research into this would be fruitful. Next steps in this research journey could be to acquire operational data from different assets to generate a probability possibility density function in order to estimate a confidence level.

REFERENCES

- [1] N.-H. Kim, D. An, and J.-H. Choi, *Prognostics and Health Management Of Engineering Systems*. Cham, Switzerland: Springer, 2017, pp. 1–24, doi: 10.1007/978-3-319-44742-1_1.
- [2] Y. Peng, M. Dong, and M. J. Zuo, “Current status of machine prognostics in condition-based maintenance: A review,” *Int. J. Adv. Manuf. Technol.*, vol. 50, nos. 1–4, pp. 297–313, Jan. 2010, doi: 10.1007/S00170-009-2482-0.
- [3] Health and Safety Executive. (Mar. 2003). Train Derailment at Potters Bar—10 May 2002—A Progress Report by the HSE Investigation Board May 2003. [Online]. Available: <https://webarchive.nationalarchives.gov.uk/ukgwa/20081108234009/http%3A//www.railreg.gov.uk/upload/pdf/incident-pottersbar-may03progrep.pdf>
- [4] G. Vachtsevanos, F. Lewis, M. Roemer, A. Hess, and B. Wu, “Fault Prognosis,” in *Intelligent Fault Diagnosis and Prognosis for Engineering Systems*, Hoboken, NJ, USA: Wiley, 2007, pp. 284–354, doi: 10.1002/9780470117842.ch6.
- [5] O. F. Eker, F. Camci, and I. K. Jennions, “A new hybrid prognostic methodology,” *Int. J. Prognostics Health Manag.*, vol. 10, no. 2, Jun. 2019, doi: 10.36001/ijphm.2019.v10i2.2727.
- [6] L. Liao and F. Köttig, “Review of hybrid prognostics approaches for remaining useful life prediction of engineered systems, and an application to battery life prediction,” *IEEE Trans. Rel.*, vol. 63, no. 1, pp. 191–207, Mar. 2014, doi: 10.1109/TR.2014.2299152.
- [7] E. Zio and G. Peloni, “Particle filtering prognostic estimation of the remaining useful life of nonlinear components,” *Reliab. Eng. Syst. Saf.*, vol. 96, no. 3, pp. 403–409, 2011, doi: 10.1016/J.RESS.2010.08.009.
- [8] D. An, J.-H. Choi, and N. H. Kim, “Prognostics 101: A tutorial for particle filter-based prognostics using MATLAB,” *Rel. Eng. Syst. Saf.*, vol. 115, pp. 161–169, Jul. 2013, doi: 10.1016/J.RESS.2013.02.019.
- [9] M. J. Daigle and K. Goebel, “Model-based prognostics with concurrent damage progression processes,” *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 43, no. 3, pp. 535–546, May 2013, doi: 10.1109/TSMCA.2012.2207109.
- [10] S. Butler and J. Ringwood, “Particle filters for remaining useful life estimation of abatement equipment used in semiconductor manufacturing,” in *Proc. Conf. Control Fault-Tolerant Syst. (SysTol)*, Oct. 2010, pp. 436–441, doi: 10.1109/SYSTOL.2010.5675984.

- [11] M. Daigle and K. Goebel, “Model-based prognostics under limited sensing,” in *Proc. IEEE Aerosp. Conf.*, Mar. 2010, pp. 1–12, doi: 10.1109/AERO.2010.5446822.
- [12] M. Samie, S. Perinpanayagam, A. Alghassi, A. M. S. Motlagh, and E. Kapetanios, “Developing prognostic models using duality principles for DC-to-DC converters,” *IEEE Trans. Power Electron.*, vol. 30, no. 5, pp. 2872–2884, May 2015, doi: 10.1109/TPEL.2014.2376413.
- [13] X.-S. Si, W. Wang, C.-H. Hu, and D.-H. Zhou, “Remaining useful life estimation—A review on the statistical data driven approaches,” *Eur. J. Oper. Res.*, vol. 213, no. 1, pp. 1–14, Aug. 2011, doi: 10.1016/J.EJOR.2010.11.018.
- [14] P. Wang and G. Vachtsevanos, “Fault prognostics using dynamic wavelet neural networks,” *Artif. Intell. Eng. Des., Anal. Manuf.*, vol. 15, no. 4, pp. 349–365, Sep. 2001, doi: 10.1017/S0890060401154089.
- [15] K. Pugalenth, H. Park, S. Hussain, and N. Raghavan, “Hybrid particle filter trained neural network for prognosis of lithium-ion batteries,” *IEEE Access*, vol. 9, pp. 135132–135143, 2021, doi: 10.1109/ACCESS.2021.3116264.
- [16] J. Yan, M. Koç, and J. Lee, “A prognostic algorithm for machine performance assessment and its application,” *Prod. Planning Control*, vol. 15, no. 8, pp. 796–801, Dec. 2004, doi: 10.1080/09537280412331309208.
- [17] M. Xia, T. Li, T. Shu, J. Wan, C. W. de Silva, and Z. Wang, “A twostage approach for the remaining useful life prediction of bearings using deep neural networks,” *IEEE Trans. Ind. Informat.*, vol. 15, no. 6, pp. 3703–3711, Jun. 2019, doi: 10.1109/TII.2018.2868687.
- [18] A. Al-Dulaimi, S. Zabihi, A. Asif, and A. Mohammadi, “Hybrid deep neural network model for remaining useful life estimation,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2019, pp. 3872–3876, doi: 10.1109/ICASSP.2019.8683763.
- [19] L. Ren, Y. Sun, J. Cui, and L. Zhang, “Bearing remaining useful life prediction based on deep autoencoder and deep neural networks,” *J. Manuf. Syst.*, vol. 48, pp. 71–77, Jul. 2018, doi: 10.1016/j.jmsy. 2018.04.008.
- [20] J. Ma, H. Su, W.-L. Zhao, and B. Liu, “Predicting the remaining useful life of an aircraft engine using a stacked sparse autoencoder with multilayer self-learning,” *Complexity*, vol. 2018, pp. 1–13, Jul. 2018, doi: 10.1155/2018/3813029.

- [21] C. Zhang, P. Lim, A. K. Qin, and K. C. Tan, “Multiobjective deep belief networks ensemble for remaining useful life estimation in prognostics,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2306–2318, Oct. 2017, doi: 10.1109/TNNLS.2016.2582798.
- [22] Y. T. Wu, M. Yuan, S. Dong, L. Li, and Y. Liu, “Remaining useful life estimation of engineered systems using vanilla LSTM neural networks,” *Neurocomputing*, vol. 75, pp. 167–179, Jan. 2018, doi: 10.1016/J.NEUCOM.2017.05.063.
- [23] R. Kizito, P. Scruggs, X. Li, M. Devinney, J. Jansen, and R. Kress, “Long short-term memory networks for facility infrastructure failure and remaining useful life prediction,” *IEEE Access*, vol. 9, pp. 67585–67594, 2021, doi: 10.1109/ACCESS.2021.3077192.
- [24] L. Zhang and J. Zhang, “A data-driven maintenance framework under imperfect inspections for deteriorating systems using multitask learningbased status prognostics,” *IEEE Access*, vol. 9, pp. 3616–3629, 2021, doi: 10.1109/ACCESS.2020.3047928.